LiveRoom Project Usage Report

SRS-style usage and system overview

Prepared from the current repository code and documentation
Date: July 13, 2026

1. Executive Summary

LiveRoom is a realtime interview workspace for candidate and HR conversations. It supports authenticated room access, live audio/video, candidate-only AI transcription, candidate verification video uploads, HR candidate recordings, transcript editing/saving, room recovery, and super-admin observation.

```
Room = temporary realtime infrastructure
Interview = persisted product record
```

2. Project Scope

- Candidate login, room creation, and recovery rejoin.
- HR language-filtered queue and interview room joining.
- Super-admin active-room observation.
- Agora live audio/video calling.
- Deepgram-powered candidate speech transcription.
- Candidate upload and HR recording verification workflows.
- Transcript editing, clearing, replacing, and final saving.
- Candidate and HR recovery after unexpected disconnects.

3. User Roles

Role	Purpose
candidate	Creates or rejoins interview rooms and provides speech/video input
hr	Joins candidate rooms, controls transcription, reviews videos, and ends interviews
super_admin	Observes full active sessions and can reset candidate verification

Supported languages: english, tamil, hindi.

4. Services Used

Service	Usage
Supabase Auth	Login, session tracking, access tokens
Supabase Postgres	Profiles, interviews, transcripts, candidate videos, verification records
Supabase Storage	Candidate verification uploads and HR candidate recordings
Socket.IO	Realtime room events, recovery events, transcript updates, participant state
Agora RTC	Live audio/video calling
Deepgram	Live candidate speech transcription
Next.js	Frontend web application
Express.js	Backend HTTP API server
AudioWorklet	Browser candidate audio processing into PCM

5. Technology Stack

Frontend: Next.js 16.2.5, React 19.2.4, TypeScript, Tailwind CSS, Framer Motion, Socket.IO Client, Agora RTC SDK, Supabase JS Client.

Backend: Node.js, Express 4.21.0, Socket.IO 4.7.5, Supabase JS Client, Agora Access Token, Deepgram SDK, CORS, dotenv.

Database and storage: Supabase Postgres, Supabase Auth, Supabase Storage.

6. High-Level Architecture

```
Browser Client
  Next.js, Supabase auth client, Socket.IO client,
  Agora RTC SDK, AudioWorklet
        |
        | HTTPS and WebSocket
        v
Node/Express Backend
  REST APIs, Socket.IO server, Supabase service-role client,
  Agora token generator, Deepgram live client,
  In-memory room store
        |
        +--> Supabase Auth
        +--> Supabase Postgres
        +--> Supabase Storage
        +--> Agora RTC Cloud
        +--> Deepgram Streaming API
```

7. Main User Workflows

Workflow	Steps
Candidate	Login, choose language, join room, wait for HR, join Agora call, upload verification video if needed, speak while HR controls transcription, rejoin recovery room if disconnected.
HR	Login, view language-matched candidate queue, join room, start/stop transcription, review videos, record candidate if needed, approve/dismiss video, save transcript, end interview.
Super Admin	Login, view active/recovering rooms, observe full sessions, reset candidate verification if required.

8. REST API Summary

The backend currently exposes 13 REST APIs: 6 GET endpoints and 7 POST endpoints.

Method	Endpoint	Purpose
GET	/	Health check
GET	/api/me	Return authenticated user and authorized role
GET	/api/token	Generate Agora RTC token
GET	/api/rooms	Return room queue/list for HR/admin
GET	/api/candidate/recovery-room	Return candidate recovery room if HR is waiting
GET	/api/candidate-videos/state	Return candidate video/upload state
POST	/api/candidate-videos/init-upload	Initialize candidate verification upload
POST	/api/candidate-videos/hr-recording/init-upload	Initialize HR recording upload
POST	/api/candidate-videos/:videoId/cancel-upload	Cancel/archive candidate upload
POST	/api/candidate-videos/:videoId/complete-upload	Mark uploaded video ready for review

Method	Endpoint	Purpose
POST	/api/candidate-videos/:videoId/approve	Approve candidate verification video
POST	/api/candidate-videos/:videoId/dismiss	Dismiss candidate verification video
POST	/api/admin/candidate/:candidateId/reset-verification	Admin reset of candidate verification

9. Socket.IO Realtime API Summary

Socket.IO is used for realtime collaboration. The project has 16 major client-emitted socket commands and 21+ server-emitted event types.

```
Client commands:
candidate-create-room, join-room, start-transcription,
stop-transcription, save-final-transcript, audio-chunk,
toggle-mute, toggle-video, transcript-edit, clear-transcript,
transcript-replace, hr-keep-waiting, hr-end-interview,
end-interview, leave-room, disconnect

Server events:
join-ack, join-error, room-state, force-logout,
room-closed, candidate-video-updated, transcription-starting,
countdown-tick, transcription-stopped, interview-ended,
block-update, transcript-saved, transcript-save-error,
hr-recovering, hr-recovery-tick, hr-rejoined,
candidate-recovering, candidate-recovery-tick,
candidate-recovery-timeout, candidate-rejoined, room-recovered
```

10. Authentication And Authorization

- Protected REST APIs require Authorization: Bearer <token>.
- Socket.IO connects with token in auth.token.
- Backend validates the Supabase token.
- Backend reads role from profiles.
- Frontend-submitted role values are not trusted.
- Agora tokens are generated only by the backend.

11. Room Lifecycle And Recovery

Room states include waiting, active, transcribing, paused, hr_recovering, candidate_recovering, waiting_for_candidate, abandoned, ending, and ended.

Candidate recovery preserves lastCandidateUser and lets the candidate return to the same open room if HR is waiting. HR recovery gives the session a grace window after unexpected HR disconnects.

12. Transcription Usage

```
Candidate microphone -> AudioWorklet -> Int16 PCM -> Socket.IO audio-chunk -> Backend Deepgram stream -> b
```

Only candidate audio is transcribed. HR starts and stops transcription but HR audio is not sent to Deepgram.

13. Candidate Verification Video Usage

Candidate verification supports local preview before upload. The selected file is not uploaded until the candidate clicks Save Upload.

- Allowed MIME types: video/webm and video/mp4.

- Maximum file size: 50 MB.
- Upload rows begin as uploading.
- Completed videos become enr and are ready for HR review.
- Approved videos become anr and are linked to candidate_verification.
- Dismissed, reset, or superseded videos become archived.

14. Database Summary

Table	Purpose
profiles	User role, language, display name
interviews	Persisted interview session
transcript_blocks	Saved transcript blocks
candidate_videos	Candidate video upload/recording history
candidate_verification	Final active verification record per candidate

Database functions: approve_candidate_video and reset_candidate_verification.

15. Storage Structure

```
Bucket: candidate-videos

candidate_user_id/candidate-upload/U_video_id.webm
candidate_user_id/hr-recording/R_video_id.webm
```

16. Important Frontend Files

File	Purpose
client/src/app/login/page.tsx	Login page
client/src/app/page.tsx	Main orchestrator
client/src/components/Lobby.tsx	Lobby, queues, candidate rejoin
client/src/components/RoomPage.tsx	Main room UI
client/src/components/TranscriptPanel.tsx	Candidate speech log and transcript controls
client/src/components/CandidateVideoPanel.tsx	Candidate upload and HR recording workflow
client/src/hooks/useSocket.ts	Socket.IO client lifecycle
client/src/hooks/useAgora.ts	Agora audio/video lifecycle
client/src/hooks/useDeepgram.ts	Candidate audio pipeline to backend
client/public/audio-processor.js	AudioWorklet PCM conversion

17. Environment Variables

```
Frontend:
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
NEXT_PUBLIC_SOCKET_URL
NEXT_PUBLIC_AGORA_APP_ID

Backend:
```

```
PORT
NEXT_PUBLIC_SUPABASE_URL
SUPABASE_SERVICE_ROLE_KEY
AGORA_APP_ID
AGORA_APP_CERTIFICATE
DEEPGRAM_API_KEY
```

18. Current Limitations And Risks

- Live room state is in memory.
- Server restart clears active rooms.
- Multi-instance deployment needs Redis or equivalent shared room state.
- Duplicate-login coordination is currently process-local.
- Supabase Storage bucket rules must match backend file limits.

19. Verification Commands

```
cd server
node --check index.js

cd client
npx tsc --noEmit
npm run lint
```

20. Final Summary

LiveRoom is a realtime interview platform powered by Next.js, Express, Socket.IO, Supabase, Agora, and Deepgram.

```
13 REST APIs
16 client socket commands
21+ server realtime events
5 main database tables
2 database functions
3 user roles
3 supported languages
```